FCEFyN
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

Programación Concurrente y Paralela

Trabajo Práctico

N.º 1

Granja de impresión 3D concurrente

Grupo: deadlocks
Institución: FCEFyN – UNC
Repositorio: `github.com/Gonzalo-Penaloza/deadlocks-tp1`

Integrantes

Fernández, Ramiro
Pandolfi, Luigi
Peñaloza, Gonzalo

Octubre de 2026

1 Introducción

El trabajo simula una granja de impresión 3D que procesa 500 órdenes. Cada orden pasa por cuatro etapas: asignación de una impresora, validación del modelo, impresión y control de calidad. Cada etapa la ejecutan varios hilos al mismo tiempo, y todas las etapas funcionan en paralelo, como una línea de producción (pipeline).

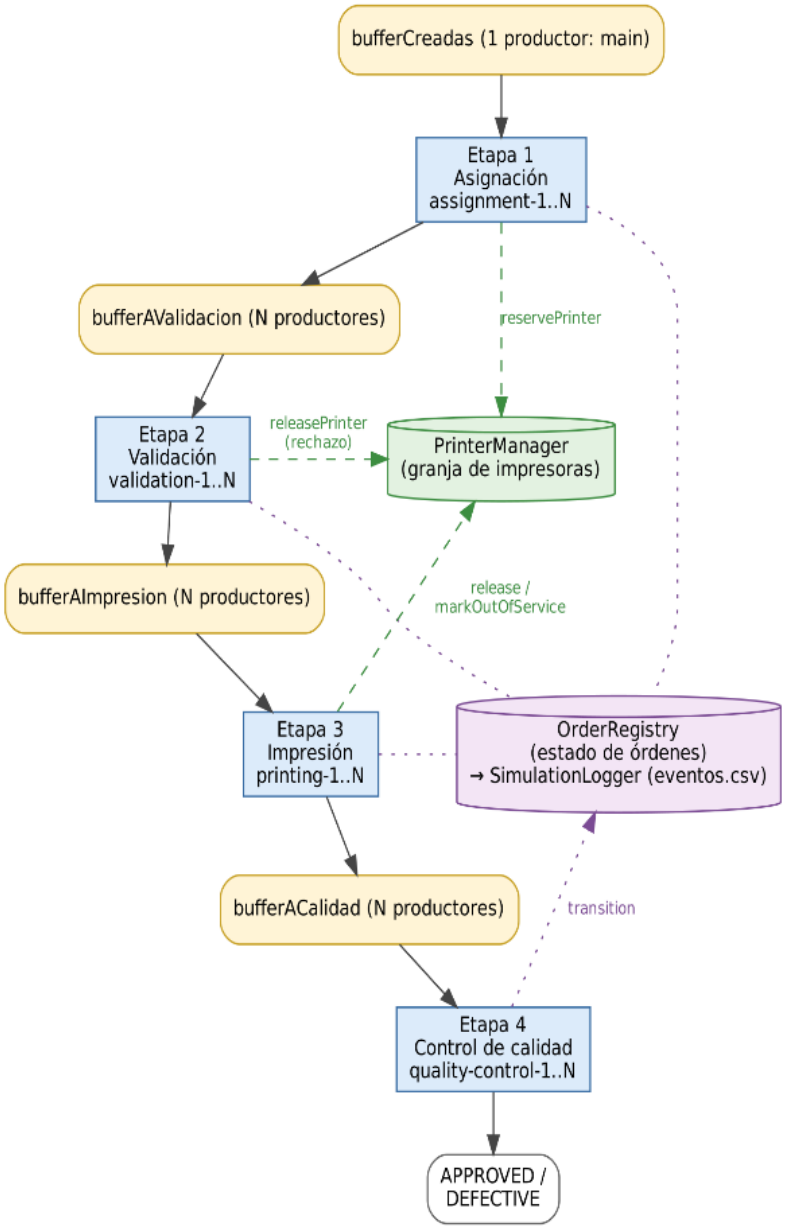


Figure 1: *Arquitectura de la solución: cuatro etapas conectadas por buffers, con dos recursos compartidos transversales (la granja de impresoras y el registro de órdenes).*

La solución está en el paquete `ar.edu.unc.fcefyn.pcp.tp1.solution`. Se implementó con Java 21 y Maven. La sincronización usa solamente `synchronized`, `wait()` y `notifyAll()`.

Cada etapa toma una orden de su buffer de entrada, la procesa y la deja en el buffer de la etapa siguiente. Las órdenes que terminan en un estado final (`REJECTED`, `PRINT_FAILED`, `APPROVED` o `DEFECTIVE`) ya no pasan a la etapa siguiente.

Etapa	Hilos	Demora	Resultado posible	Impresora
1. Asignación	`assignment-1..3`	50 ms	`CREATED` → `WAITING_VALIDATION`	La reserva
2. Validación	`validation-1..2`	120 ms	→ `READY_TO_PRINT` o → `REJECTED`	La libera si se rechaza
3. Impresión	`printing-1..3`	180 ms	→ `PRINTED` o → `PRINT_FAILED`	La libera, o queda `OUT_OF_SERVICE` si falla
4. Calidad	`quality-control-1..2`	80 ms	→ `APPROVED` o → `DEFECTIVE`	No usa

Table 1: *Resumen de etapas, trabajadores, tiempos de procesamiento y gestión de recursos.*

2 Diseño de la solución

2.1 Clases principales

Clase	Responsabilidad	¿Es monitor?
`ConcurrentSimulation`	Crea los recursos, los buffers y los hilos; los arranca, espera con `join()` y arma el resultado.	No
`OrderBuffer`	Cola acotada productor/consumidor entre dos etapas. Lleva la cuenta de cuántos productores siguen activos.	Sí
`PrinterManager`	Administra la matriz de 10×20 impresoras: reservar, liberar y marcar fuera de servicio.	Sí
`Printer`	Datos de una impresora (estado, usos, orden asignada). Solo se accede desde `PrinterManager`.	No (protegida)
`OrderRegistry`	Estado de las 500 órdenes. Cambia el estado y escribe el log en una sola operación atómica.	Sí
`Order`	Datos de una orden (estado, impresora, contadores por etapa). Verifica que cada transición sea válida.	No (protegida)
`SimulationLogger`	Escribe `eventos.csv` con número de secuencia y tiempo transcurrido.	Sí
`*Worker`	El trabajo de cada etapa (`Runnable`).	No
`ResultsCollector`	Escribe `elementos.csv` y `resumen.properties` al final.	No

Table 2: *Clases principales del sistema y su carácter de monitor.*

2.2 Diagrama de clases

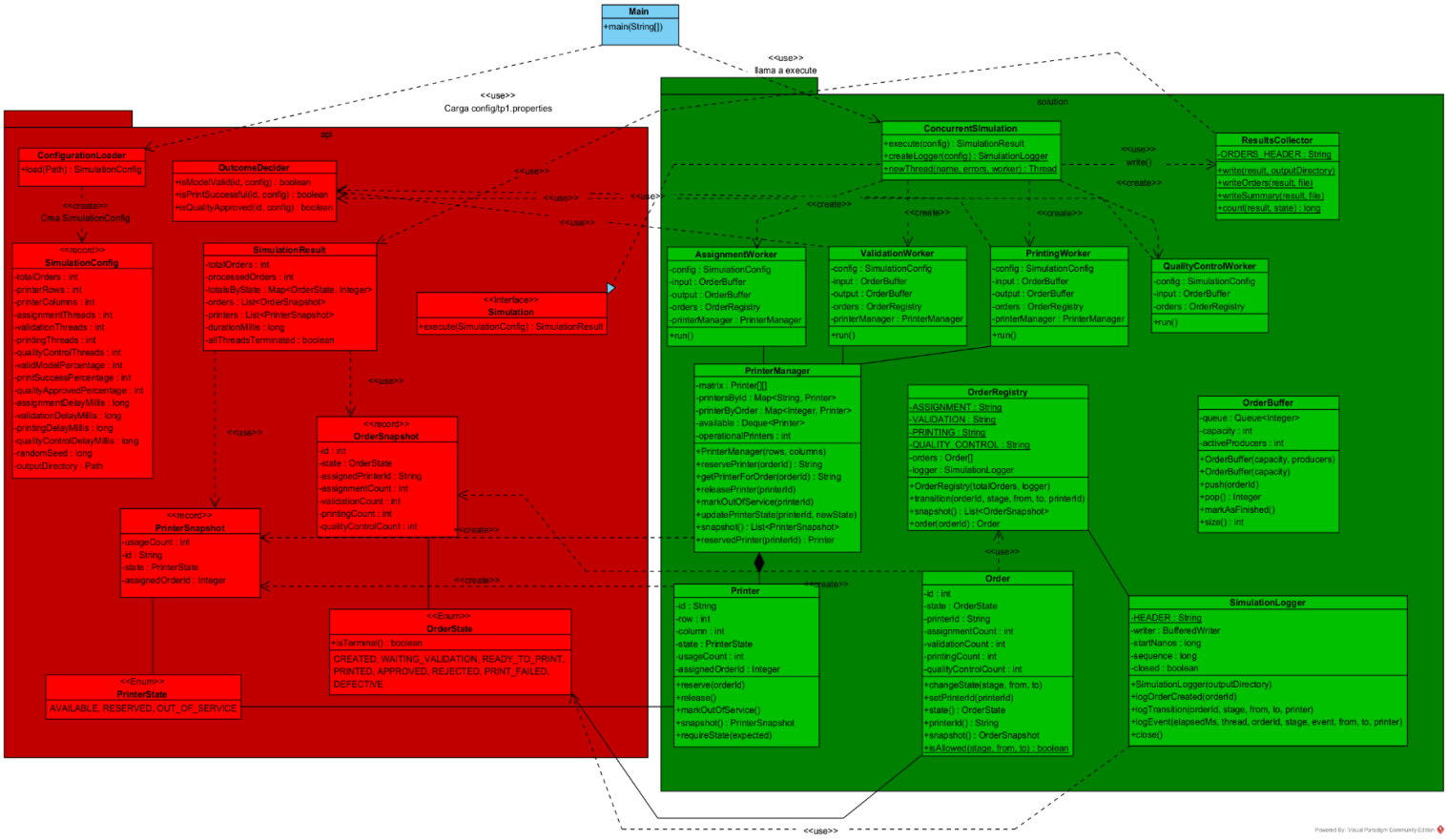


Figure 2: *Diagrama de clases en capa API y Solution.*

El diagrama separa dos paquetes. En `api` (rojo) están las clases que da la cátedra: la interfaz `Simulation`, la configuración `SimulationConfig`, los records de resultado (`SimulationResult`, `OrderSnapshot`, `PrinterSnapshot`), los enumerados de estados y `OutcomeDecider`, que decide de forma determinística si una orden se valida, se imprime bien o se aprueba. En `solution` (verde) está lo que implementó el grupo.

Las relaciones más importantes son:

- `ConcurrentSimulation` implementa `Simulation`, crea los cuatro tipos de worker y usa `ResultsCollector` para escribir los resultados.
- Cada worker conoce su `OrderBuffer` de entrada y de salida, el `OrderRegistry` y (salvo calidad) el `PrinterManager`. Los workers no tienen datos compartidos propios: todo lo compartido está en los monitores.
- `PrinterManager` se compone de objetos `Printer` (rombo lleno): las impresoras solo existen dentro del administrador y nadie más las modifica. Lo mismo pasa entre `OrderRegistry` y `Order`.
- `OrderRegistry` usa a `SimulationLogger` para registrar cada transición.

3 Recursos compartidos

Un recurso compartido es un dato al que acceden dos o más hilos, y donde al menos uno de ellos lo modifica. Estos son todos los recursos compartidos de la solución:

- **Los 4 `OrderBuffer` (`queue`, `activeProducers`):** La etapa anterior deposita (`push`) y la siguiente retira (`pop`). El primer buffer lo llena `main`. Protege que dos consumidores no saquen la misma orden, que no se pierdan órdenes y que el contador de productores sea exacto.
- **`PrinterManager` (matriz, cola `available`, mapas, `operationalPrinters`) y cada `Printer`:** Asignación reserva; validación libera; impresión libera o deja fuera de servicio. Protege que una impresora no se reserve para dos órdenes, y que el estado, los usos y la orden asignada sean coherentes.
- **`OrderRegistry` y cada `Order` (estado, impresora, contadores):** Las cuatro etapas cambian estados. Protege que cada transición parta del estado esperado y que los contadores por etapa no pierdan incrementos.
- **`SimulationLogger` (`writer`, `sequence`):** Todas las etapas, a través de `OrderRegistry`. Protege que las líneas de `eventos.csv` no se mezclen y que los números de secuencia sean únicos y crecientes.
- **Lista `errors` de `ConcurrentSimulation`:** Cualquier hilo que termine con una excepción. Protege que no se pierdan errores (se protege con `synchronized (errors)`).

`SimulationConfig` también lo leen todos los hilos, pero es un record inmutable: como nadie lo modifica, no hace falta sincronizarlo.

4 Condiciones de carrera posibles

Una condición de carrera ocurre cuando el resultado depende del orden en que se intercalan los hilos. Pasa cuando dos hilos hacen "leer – decidir – escribir" sobre el mismo dato y el planificador los interrumpe a la mitad. Estas son las que existirían sin sincronización:

a) **Dos hilos de asignación reservan la misma impresora:** Si dos hilos leen que la impresora P-0-0 está disponible al mismo tiempo, ambos la reservarán para órdenes distintas. La orden 1 cree que tiene la impresora, pero la impresora dice que es de la orden 2. Además `usageCount` puede quedar en 1 en vez de 2. *Solución:* `reservePrinter` es `synchronized`, así que buscar y reservar es una única operación indivisible.

b) **Dos consumidores sacan la misma orden del buffer:** Con una `LinkedList` sin protección, dos `poll()` simultáneos pueden devolver el mismo elemento o romper los enlaces internos de la lista: una orden se procesaría dos veces y otra se perdería. *Solución:* `push` y `pop` son `synchronized`.

c) **Contadores que pierden incrementos:** `usageCount++` y los contadores de etapa de `Order` son en realidad tres pasos (leer, sumar, escribir). Si dos hilos leen el mismo valor, ambos escriben valor $+ 1$ y se pierde un incremento. *Solución:* solo se modifican dentro de métodos `synchronized` del monitor dueño.

d) **Log desordenado o inconsistente con el estado:** Si un hilo cambia el estado y otro escribe su línea de log entre medio, `eventos.csv` puede mostrar las transiciones en un orden que nunca ocurrió. También, si la secuencia no está protegida, dos líneas pueden tener el mismo número. *Solución:* `OrderRegistry.transition` cambia el estado y escribe el log dentro del mismo método `synchronized`; `SimulationLogger` además tiene sus métodos sincronizados.

e) **Liberar la impresora antes de registrar la transición:** Si impresión liberara la impresora y después registrara `PRINTED`, otro hilo podría reservarla y loguear su `ASSIGNMENT` antes que la liberación quede registrada: en el log la impresora aparecería usada por dos órdenes a la vez. *Solución:* se respeta la regla "reservar y después registrar; registrar y después liberar".

f) **Terminación prematura o tardía:** Si un consumidor mirara "¿el buffer está vacío?" y "¿terminaron los productores?" en dos momentos distintos, podría irse justo antes de que llegue la última orden (se pierde) o quedarse dormido para siempre (el programa no termina). *Solución:* las dos condiciones se evalúan juntas dentro de `pop()`, que es `synchronized`.

5 Estrategia de sincronización

Todos los recursos compartidos se protegen con el patrón monitor. Un monitor funciona como una habitación con una sola llave: para ejecutar cualquiera de sus métodos `synchronized`, un hilo tiene que tomar la llave (el lock). Mientras la tenga, ningún otro hilo puede entrar, y los que llegan esperan en la puerta. Al salir del método, el hilo devuelve la llave, y Java la devuelve sola aunque haya ocurrido una excepción.

Hay cuatro monitores: cada `OrderBuffer`, `PrinterManager`, `OrderRegistry` y `SimulationLogger`. Cada uno tiene su propia llave: el propio objeto (`this`). Por eso, que un hilo esté usando la granja de impresoras no impide que otro use un buffer.

Dentro de un monitor, un hilo que no puede avanzar llama a `wait()`: suelta la llave y se duerme hasta que otro hilo lo despierta con `notifyAll()`.

5.1 Por qué elegimos cada mecanismo

- **`synchronized` y no `ReentrantLock`:** Para que un hilo pueda esperar usando `ReentrantLock` hace falta `Condition`, y el enunciado lo prohíbe. `synchronized` trae la espera incorporada (`wait()`/`notifyAll()`) y además devuelve el lock automáticamente: no hay riesgo de olvidarse un `unlock()`.

- **`notifyAll()` y no `notify()`:** En la sala de espera de un mismo buffer puede haber hilos que esperan cosas distintas: productores que esperan lugar y consumidores que esperan órdenes. `notify()` despierta a un solo hilo, elegido al azar. Si llega una orden y se despierta un productor, ese productor no puede hacer nada con eso y se vuelve a dormir; el consumidor que sí la necesitaba nunca se entera y el programa puede quedar trabado. `notifyAll()` despierta a todos, y cada uno revisa si lo que espera está listo.

- **`wait()` dentro de `while` y no de `if`:** Que a un hilo lo despierten no garantiza que lo que esperaba siga disponible. Por ejemplo, hay una sola orden en el buffer y se despiertan dos consumidores. El primero toma la llave y se lleva la orden; cuando entra el segundo, el buffer ya está vacío otra vez. Con `if`, el segundo no vuelve a mirar y saca `null` de una cola vacía. Con `while`, vuelve a preguntar, ve que no hay nada y se vuelve a dormir. El `while` también cubre los despertares espurios: Java a veces despierta un hilo sin que nadie lo haya notificado.

- **Sin espera activa:** Ningún hilo pregunta en un ciclo "¿ya hay algo?", gastando CPU sin hacer nada útil. Mientras espera, el hilo está dormido en `wait()` y no consume procesador hasta que lo despiertan.

5.2 Por qué no puede haber interbloqueo

Un interbloqueo (*deadlock*) ocurre cuando dos hilos se esperan mutuamente para siempre. Por ejemplo, el hilo A tiene la llave del monitor 1 y espera la del monitor 2, mientras el hilo B tiene la del 2 y espera la del 1. Como ninguno suelta la suya, ninguno avanza. Para que esto pase, hacen falta dos condiciones a la vez:

1. Que un hilo tenga una llave mientras pide otra.
2. Que esos pedidos formen un círculo.

En esta solución:

- Los workers usan un monitor por vez: entran a `PrinterManager` y salen; después entran a `OrderRegistry` y salen; después al buffer y salen. Nunca tienen dos llaves al mismo tiempo.
- El único caso en que un hilo tiene dos llaves es `OrderRegistry.transition`, que adentro llama a `SimulationLogger.logTransition`. El orden es siempre el mismo (primero el registro, después el logger), y el logger nunca llama al registro. No hay círculo.
- `wait()` solo se llama dentro de un monitor y suelta la llave de ese monitor. Ningún hilo se queda dormido con la llave de otro monitor.

Como nunca se forma un círculo de espera, el interbloqueo es imposible.

6 Condiciones de espera y despertar

Monitor	Quién espera	Condición de espera (`while`)	Quién lo despierta
`OrderBuffer.push`	Productor (etapa anterior)	`queue.size() == capacity`	`pop()` con `notifyAll()` al sacar una orden
`OrderBuffer.pop`	Consumidor (etapa siguiente)	`queue.isEmpty() && activeProducers > 0`	`push()` al agregar una orden, o `markAsFinished()` cuando termina el último productor
`PrinterManager.reservePrinter`	Hilos de asignación	`available.isEmpty()` (y quedan impresoras operativas)	`releasePrinter()` (validación o impresión) o `markOutOfService()`

Table 3: *Mapeo explícito de esperas y despertares.*

Listing 1: *Implementación del método pop() con wait() dentro de while.*

```
public synchronized Integer pop() throws InterruptedException {
    while (queue.isEmpty() && activeProducers > 0) {
        wait();                 // suelta el lock y duerme
    }
    if (queue.isEmpty()) {
        return null;            // vacio y sin productores: la etapa
            termino
    }
    Integer order = queue.poll();
```

```
    notifyAll();                // puede haber productores esperando lugar
    return order;
}
```

En la práctica, el buffer de capacidad 500 nunca se llena, porque nunca hay más de 500 órdenes. El límite real lo pone la granja: hay solo 200 impresoras y cada orden retiene la suya desde la asignación hasta que se rechaza o termina de imprimirse. Por eso la espera que más se observa es la de los hilos de asignación en `reservePrinter`.

7 Mecanismo de terminación

Los hilos no se interrumpen desde afuera: cada etapa termina sola, en cascada.

1. `main` deposita las 500 órdenes en `bufferCreadas` y llama a `markAsFinished()`. Ese buffer tenía un solo productor, así que queda cerrado.

2. Cada `OrderBuffer` sabe cuántos productores tiene (por ejemplo, `bufferAValidacion` tiene 3, uno por hilo de asignación). Cuando un hilo de una etapa recibe `null` de su `pop()`, sale del ciclo y, en el bloque `finally`, llama a `markAsFinished()` sobre su buffer de salida.

3. Cuando el último productor de un buffer termina, `activeProducers` llega a 0 y `markAsFinished()` hace `notifyAll()`. Los consumidores dormidos se despiertan, ven el buffer vacío sin productores y reciben `null`. Así el cierre se propaga a la etapa siguiente.

4. `main` espera a todos los hilos con `join()`, cierra el logger y escribe los resultados.

`markAsFinished()` está en un `finally` para que se ejecute siempre, aunque el hilo termine por una excepción. Si no, la etapa siguiente esperaría para siempre a un productor que ya no existe.

`join()` además garantiza visibilidad: todo lo que un hilo escribió antes de terminar es visible para quien hizo `join()` sobre él (relación *happens-before*). Por eso `main` puede leer los estados finales con seguridad. Al final se verifica que las 500 órdenes estén en estado final, que los buffers estén vacíos y que ningún hilo haya terminado con error. Si algo falla, se lanza una excepción en lugar de devolver un resultado incompleto.

8 Análisis teórico del tiempo de ejecución

Las demoras se simulan con `Thread.sleep`, así que el tiempo lo determinan las demoras y la cantidad de hilos, no la velocidad del procesador. Con la semilla oficial, `OutcomeDecider` produce siempre los mismos resultados: 71 órdenes rechazadas en validación, 45 fallas de impresión y 25 piezas defectuosas. Eso define cuántas órdenes llegan a cada etapa:

Etapa	Órdenes	Hilos	Demora	Trabajo de la etapa = ⌈órdenes/hilos⌉ × demora
Asignación	500	3	50 ms	$\lceil 500/3 \rceil \times 50 = 167 \times 50 = 8.350$ ms
Validación	500	2	120 ms	$\lceil 500/2 \rceil \times 120 = 250 \times 120 = 30.000$ ms
Impresión	$500 - 71 = 429$	3	180 ms	$\lceil 429/3 \rceil \times 180 = 143 \times 180 = 25.740$ ms
Calidad	$429 - 45 = 384$	2	80 ms	$\lceil 384/2 \rceil \times 80 = 192 \times 80 = 15.360$ ms

Table 4: *Desglose detallado del tiempo de trabajo por etapa.*

Tres escenarios de referencia:

- **Todo secuencial (un solo hilo hace todo):** $500 \cdot 50 + 500 \cdot 120 + 429 \cdot 180 + 384 \cdot 80 = 25.000 + 60.000 + 77.220 + 30.720 = 192.940$ ms $\approx 3,2$ min.
- **Etapas una después de otra, con sus hilos, pero sin solaparse:** $8.350 + 30.000 + 25.740 + 15.360 = 79.450$ ms.
- **Pipeline (lo implementado):** como las cuatro etapas trabajan a la vez, el tiempo total lo marca la etapa más lenta, el cuello de botella. Acá es validación (30.000 ms). A eso se suma lo que tarda la primera orden en llegar a validación (50 ms de asignación) y lo que tarda la última orden validada en salir del sistema (180 ms de impresión +80 ms de calidad):

  $T_{\text{teórico}} \approx$ trabajo del cuello de botella+demoras de las otras etapas $= 30.000+50+180+80 = 30.310$ ms

Speedup y eficiencia: El speedup es cuánto más rápido es el programa concurrente que el secuencial: $S = T_{\text{secuencial}}/T_{\text{concurrente}}$. La eficiencia divide el speedup por la cantidad de hilos: $E = S/p$ ($p =$ cantidad de hilos).

- Con la configuración oficial (10 hilos): $S = 192.940/30.439 = 6,34$ y $E = 6,34/10 = 63\%$.
- Con un hilo por etapa (4 hilos): $S = 192.940/77.571 = 2,49$. Esa mejora se debe solo a que las etapas se solapan.

La eficiencia no llega al 100% porque las etapas no están balanceadas: mientras validación trabaja a pleno, los hilos de asignación y de calidad pasan buena parte del tiempo esperando. La Ley de Amdahl dice que la parte que no se puede paralelizar limita la mejora total. En un pipeline, el papel de esa parte lo cumple el cuello de botella: agregar hilos a una etapa que no es el cuello de botella no reduce el tiempo total. Como los hilos pasan la mayor parte del tiempo en `sleep()` y no calculando, la cantidad de núcleos de la máquina casi no influye.

Por qué la asignación termina a los ~ 20 s y no a los $8,35$ s: Asignación sola podría terminar en $8,35$ s, pero cada orden retiene su impresora hasta que se rechaza o termina de imprimirse. Las 200 impresoras se agotan antes de que validación termine, y desde ese momento asignación solo avanza cuando se libera alguna. En los registros se ve que llega a haber 196 impresoras reservadas a la vez: todas las que siguen operativas. Es decir, la granja funciona como un buffer acotado entre asignación y el resto de la línea.

9 Resultados de múltiples ejecuciones

Se ejecutó la configuración oficial (`config/tp1.properties`: 500 órdenes, 10×20 impresoras, hilos 3/2/3/2, demoras 50/120/180/80 ms, semilla 20261001) cinco veces seguidas en la misma JVM:

Corrida	Duración (ms)	Aprob.	Rech.	Fall.	Defect.	Última asignación	Fuera de serv.
oficial-1	30.504	359	71	45	25	20.263 ms	45
oficial-2	30.436	359	71	45	25	20.197 ms	45
oficial-3	30.421	359	71	45	25	20.184 ms	45
oficial-4	30.420	359	71	45	25	20.183 ms	45
oficial-5	30.415	359	71	45	25	20.180 ms	45

Table 5: *Métricas obtenidas en 5 ejecuciones consecutivas.*

- **Duración:** promedio 30.439 ms, desvío estándar 37 ms, mínimo 30.415 ms, máximo 30.504 ms. La primera corrida es la más lenta, porque la JVM todavía está cargando clases y compilando código (calentamiento).
- **Resultados de negocio idénticos en todas las corridas:** 359 aprobadas + 71 rechazadas + 45 fallidas + 25 defectuosas = 500. Esto es lo esperado, porque `OutcomeDecider` depende solo de la semilla, del número de orden y de la etapa, no del orden en que se ejecutan los hilos. Lo que sí cambia entre corridas es el intercalado: qué hilo procesa cada orden y qué impresora le toca.
- 45 impresoras fuera de servicio, una por cada falla de impresión.
- La ejecución oficial que está en `resultados/` se hizo en la PC de un integrante (Windows, desde IntelliJ) y tardó 31.545 ms. Con el `.jar` empaquetado tardó 31.580 ms. Es alrededor de 1 s más que en las corridas de la tabla. Una explicación posible es que en Windows `Thread.sleep` suele dormir un poco más de lo pedido. También puede influir escribir los archivos en una carpeta sincronizada con OneDrive.

En las 18 corridas realizadas (las de esta sección y las de la sección 11) se verificaron automáticamente, sobre `eventos.csv`, estas propiedades:

- Los números de secuencia son consecutivos y el tiempo nunca retrocede.
- Cada orden sigue un camino válido de estados y termina en exactamente un estado final.
- Ninguna impresora está reservada por dos órdenes al mismo tiempo.
- Al terminar no queda ninguna impresora reservada.
- Cada archivo tiene 2.313 eventos: 500 creaciones + 500 asignaciones + 500 validaciones + 429 impresiones + 384 controles.

10 Comparación entre tiempos teóricos y observados

Experimento	Hilos A/V/I/C	Demoras A/V/I/C (ms)	Cuello de Botella	Teórico (ms)	Observado (ms)	Dif.
Oficial (promedio de 5)	3/2/3/2	50/120/180/80	Validación	30.310	30.439	+0.4%
Validación con 1 hilo	3/1/3/2	50/120/180/80	Validación	60.310	60.412	+0.2%
Validación con 3 hilos	3/3/3/2	50/120/180/80	Impresión	25.990	26.100	+0.4%
Validación con 4 hilos	3/4/3/2	50/120/180/80	Impresión	25.990	26.097	+0.4%
Asignación con 1 hilo	1/2/3/2	50/120/180/80	Validación	30.310	30.418	+0.4%
Asignación con 6 hilos	6/2/3/2	50/120/180/80	Validación	30.310	30.416	+0.3%
Impresión con 1 hilo	3/2/1/2	50/120/180/80	Impresión	77.470	77.569	+0.1%
Impresión con 6 hilos	3/2/6/2	50/120/180/80	Validación	30.310	30.362	+0.2%
Calidad con 1 hilo	3/2/3/1	50/120/180/80	Calidad	31.070	31.217	+0.5%
1 hilo por etapa	1/1/1/1	50/120/180/80	Impresión	77.470	77.571	+0.1%
Demora de validación 60 ms	3/2/3/2	50/60/180/80	Impresión	25.930	26.041	+0.4%
Demora de impresión 360 ms	3/2/3/2	50/120/360/80	Impresión	51.730	51.885	+0.3%
Solo 50 impresoras (5 × 10)	3/2/3/2	50/120/180/80	Validación	30.310	30.461	+0.5%
Todas las demoras en 0	3/2/3/2	0/0/0/0	—	0	13	—

Table 6: *Tabla comparativa con los 14 experimentos ejecutados.*

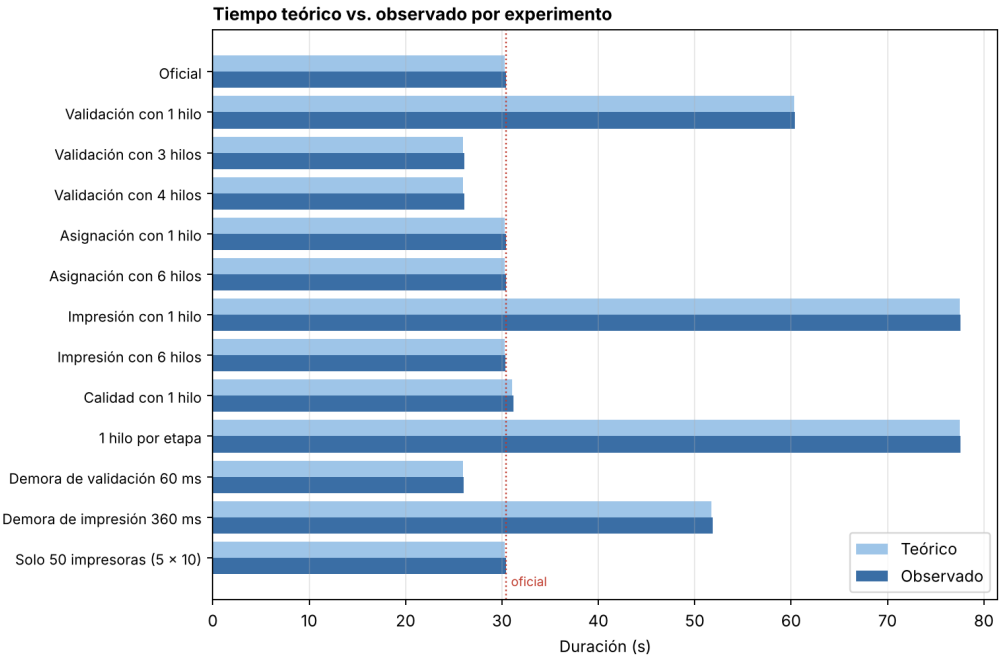


Figure 3: *Tiempo teórico vs. observado. El modelo del cuello de botella predice todos los casos con un error menor al 1%.*

En todos los casos, el tiempo observado es entre 0,1% y 0,5% mayor que el teórico. La diferencia (entre 50 y 160 ms) se explica por la sobrecarga real:

- `sleep()` nunca duerme menos de lo pedido y suele dormir un poco más.
- Despertar un hilo después de `notifyAll()` y volver a tomar el lock lleva tiempo.
- Los hilos compiten por el lock del logger y escriben en disco.
- Al final se arman los snapshots y se escriben los archivos de resultados.

Que la diferencia sea tan chica indica que las esperas por los monitores casi no frenan el programa: los hilos pasan casi todo su tiempo en las demoras simuladas.

11 Efecto de cambiar la cantidad de hilos y las demoras

Se ejecutaron 13 variantes de la configuración oficial, cambiando un parámetro por vez. Los resultados están en la tabla de la sección 10:

- **Hilos de validación (el cuello de botella):** Con 1 hilo, el tiempo se duplica (60,4 s): validación pasa a necesitar $500 \times 120 = 60$ s. Con 3 hilos baja a 26,1 s (−14%), pero no a 20 s como podría esperarse, porque el cuello de botella pasa a ser impresión (25,74 s). Por eso con 4 hilos no hay ninguna mejora: 26,1 s otra vez.
- **Hilos de asignación:** Con 1 o con 6 hilos, el tiempo es el mismo (30,4 s). Incluso con 1 hilo (25 s de trabajo), asignación sigue siendo más rápida que validación. Lo único que cambia es cuándo termina asignación (25,1 s con 1 hilo y 20,2 s con 6), porque igual tiene que esperar impresoras libres.
- **Hilos de impresión:** Con 1 hilo, impresión pasa a ser el cuello de botella: $429 \times 180 = 77{,}2$ s, y se observan 77,6 s. Con 6 hilos no mejora (30,4 s), porque el límite sigue siendo validación.
- **Hilos de calidad:** Con 1 hilo, calidad necesita $384 \times 80 = 30{,}7$ s, casi lo mismo que validación. Pasa a ser el cuello de botella por muy poco y el total sube apenas, a 31,2 s.
- **Un hilo por etapa:** 77,6 s: lo determina impresión con 1 hilo. Aun así es 2,5 veces más rápido que hacer todo en secuencia, solo gracias al pipeline.
- **Demoras:** Si la validación baja a 60 ms, el cuello de botella pasa a impresión (26,0 s). Si la impresión sube a 360 ms, impresión pasa a ser el cuello de botella con $429 \times 360/3 = 51{,}5$ s (se observan 51,9 s). Con todas las demoras en 0, la simulación tarda 13 ms: es el costo de la sincronización, el logging y la escritura de archivos, sin trabajo simulado.
- **Cantidad de impresoras:** Con solo 50 impresoras (5×10), el tiempo no cambia (30,5 s), porque validación sigue siendo el límite. Lo que cambia es que asignación queda frenada casi hasta el final (termina a los 30,1 s) y como máximo hay 49 impresoras reservadas a la vez. Al terminar, de las 50 impresoras solo 5 quedan operativas, porque 45 quedan fuera de servicio. Con todavía menos impresoras, la granja podría pasar a ser el cuello de botella. Si quedaran 0 operativas, `reservePrinter` lanza una excepción en lugar de esperar para siempre.

Conclusión: para que el programa sea más rápido, hay que agregar hilos (o bajar la demora) solo en la etapa cuello de botella, y hacerlo hasta que otra etapa la alcance. Agregar hilos en otras etapas no mejora el tiempo y suma hilos que pasan la mayor parte del tiempo esperando.

12 Diagrama de secuencia

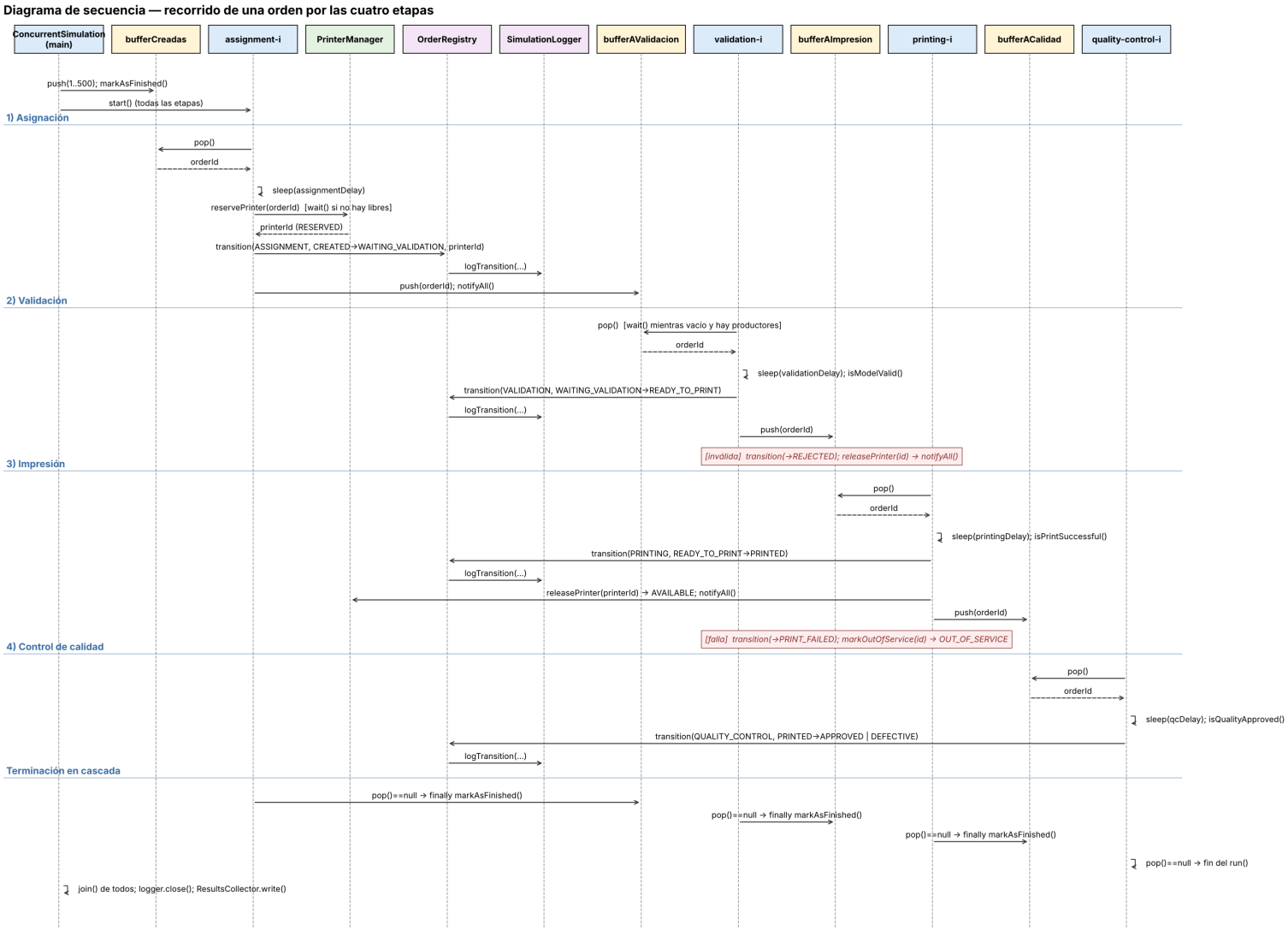


Figure 4: *Diagrama de secuencia: recorrido de una orden por las cuatro etapas y terminación en cascada.*

El diagrama sigue a una orden desde que `main` la deposita en `bufferCreadas` hasta que calidad la aprueba o la marca como defectuosa:

- **Asignación:** el hilo `assignment-i` hace `pop()` en `bufferCreadas`, duerme 50 ms y pide una impresora con `reservePrinter()`. Si no hay impresoras libres, espera en `wait()` dentro de `PrinterManager`. Con la impresora reservada, registra la transición en `OrderRegistry` (que a su vez escribe en `SimulationLogger`) y deposita la orden en `bufferAValidacion`.
- **Validación:** si el modelo es válido, la orden pasa a `READY_TO_PRINT` y sigue a impresión con la impresora todavía reservada. Si es inválido, se registra `REJECTED` y después se libera la impresora. El `notifyAll()` de `releasePrinter()` despierta a los hilos de asignación que esperaban una.
- **Impresión:** si la impresión sale bien, se registra `PRINTED`, se libera la impresora y la orden pasa a calidad. Si falla, se registra `PRINT_FAILED` y la impresora queda `OUT_OF_SERVICE` para siempre.

- **Calidad:** decide entre `APPROVED` y `DEFECTIVE`. Esta etapa no usa impresoras.
- **Terminación:** cuando un hilo recibe `null` de `pop()`, sale de su ciclo y llama a `markAsFinished()` sobre el buffer siguiente. Cuando terminan todos, `main` hace `join()`, cierra el logger y escribe los resultados.

13 Justificación de las decisiones de diseño

- **Un monitor por recurso, y no un único lock global:** Si hubiera un solo lock, un hilo que imprime y uno que valida se bloquearían entre sí aunque no compartan nada. Con monitores separados, solo compiten los hilos que de verdad usan el mismo recurso.
- **`OrderRegistry` cambia el estado y escribe el log dentro de un mismo método `synchronized`:** Así el orden de las líneas de `eventos.csv` es exactamente el orden en que ocurrieron los cambios, y no puede pasar que el estado cambie sin quedar registrado.
- **Buffers con contador de productores en lugar de "píldoras venenosas" (órdenes falsas que avisan el fin):** Las píldoras obligan a calcular cuántas mandar y pueden confundirse con órdenes reales. El contador hace que el cierre en cascada sea automático y verificable.
- **Capacidad del buffer = cantidad total de órdenes:** El enunciado no impone un límite. El recurso escaso real es la granja, que ya limita cuántas órdenes hay en proceso al mismo tiempo. Igual el buffer soporta capacidad acotada: si está lleno, `push` espera.
- **`Printer` y `Order` no son monitores:** Solo se acceden desde dentro de `PrinterManager` y `OrderRegistry`, que ya tienen el lock. Sincronizarlas otra vez agregaría locks anidados sin ningún beneficio.
- **Cola `available` (`Deque`) en `PrinterManager`:** Buscar una impresora libre cuesta $O(1)$, en lugar de recorrer la matriz de 200 impresoras cada vez mientras se tiene el lock.
- **Validación de transiciones:** `Order.changeState` verifica el estado de origen y que el camino sea válido. Si hubiera un error de sincronización, se vería como una excepción en lugar de quedar como datos corruptos.
- **Hilos con nombre (`assignment-1`, etc.):** Para que el log muestre qué hilo hizo cada cosa. Además, un manejador de excepciones no capturadas (*uncaught exception handler*) junta los errores para informarlos al final.
- **Ejecución determinística de resultados:** Gracias a `OutcomeDecider`, el resultado final (cuántas órdenes terminan en cada estado) se puede comparar entre corridas aunque el intercalado de los hilos cambie.

14 Pruebas

Además de los tests públicos de la cátedra, el grupo escribió tests para cada componente. En total hay 116 tests, y todos pasan con `mvn clean test`:

- `OrderBufferTest`: varios productores y consumidores concurrentes; se verifica que ninguna orden se pierde ni se duplica (repetido 50 veces), y que el consumidor espera hasta que termina el último productor.
- `PrinterManagerTest`: matriz inicial con todas las impresoras disponibles, que una impresora nunca quede asignada a dos órdenes a la vez, que la reserva espera hasta que se libere una impresora, y que las impresoras fuera de servicio no se reutilizan.

- `AssignmentWorkerTest`: tres hilos de asignación procesan cada orden exactamente una vez (repetido 20 veces).
- `SimulationLoggerTest` y `ResultsCollectorTest`: formato de `eventos.csv`, `elementos.csv` y `resumen.properties`, y que con varios hilos escribiendo a la vez la secuencia del log sigue siendo consecutiva.

Como la ejecución concurrente no es determinística, una sola corrida exitosa no prueba que el programa sea correcto. Por eso los tests concurrentes se repiten muchas veces (`@RepeatedTest`), y en las 18 corridas del informe se verificaron las propiedades de la sección 9.

15 Conclusiones

La solución procesa las 500 órdenes sin pérdidas, sin duplicados y sin interbloqueos. Usa solamente `synchronized`, `wait()` y `notifyAll()`, y termina sola gracias al cierre en cascada de los buffers. El tiempo de ejecución coincide con el modelo teórico del cuello de botella con un error menor al 1%. Ese modelo también permite predecir qué cambio de configuración mejora el rendimiento: en la configuración oficial, el límite es validación, y si se le agregan hilos, el límite pasa a impresión.